



CS/CE 224/272 - Object Oriented Programming and Design Methodology

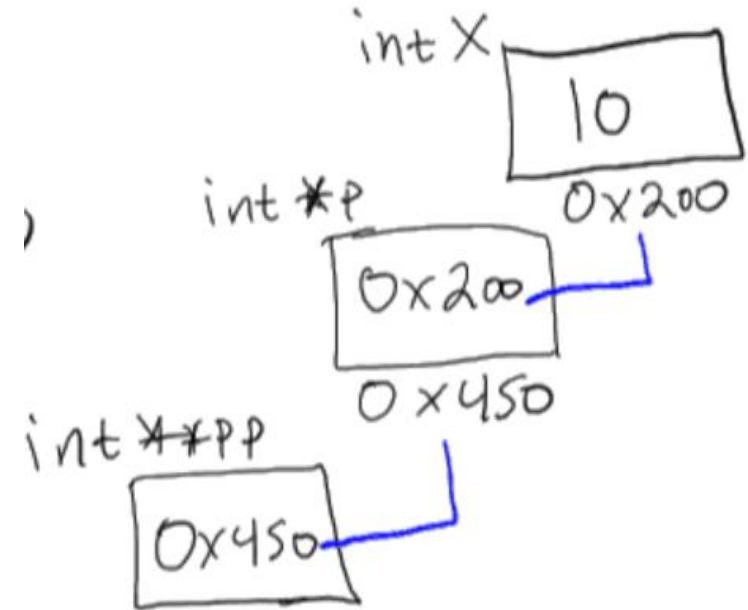
Nadia Nasir



Pointers to pointers

Pointers to pointers

- A normal pointer to an int is declared using a single asterisk:
- `int x = 10;`
- `int* p = &x;`
- A pointer to a pointer to an int is declared using two asterisks
- `int x = 10;`
- `int* p = &x;`
- `int** pp = &p;`



Example

```
• int main()
• {

    • int x{ 5 };

    • int* ptr { &x };
    • std::cout << *ptr << '\n'; // Dereference pointer to int to
      get value of variable of x

    • int** ptrptr { &ptr };
    • std::cout << **ptrptr << '\n'; // dereference to get pointer
      to int, dereference again to get value of variable x

    • return 0;
• }
```

Type of Memory Allocation

Various Types of Memory Allocation

- In order to work with data, we **allocate some memory** for it.
- There are three types of memory allocation.
 - Static (*for **static** and **global** variables*)
 - Automatic (***function parameters** and **local variables***)
 - Dynamic (*today's topic*)
- **Automatic memory allocation** – we've already been doing this.

Automatic Memory Allocation

- Happens for **function parameters** and **local variables**.
- Memory is allocated when the relevant block is entered.
- Memory is freed when the block is exited.
- Allocation/deallocation **happens automatically**.

```
1  #include<iostream>
2
3  int main()
4  {
5      int num{ 5 };
6
7      {
8          double pi{ 3.14 };
9
10         } // "pi" is inaccessible outside this block.
11
12         std::cout << pi << '\n';
13
14         return 0;
15     }
```

Memory for **num** allocated automatically when you initialized it.

Memory for **pi** allocated automatically when you initialized it.

As soon as the code block ends, **pi** goes out of scope. In other words, memory for **pi** has been automatically deallocated.

Same holds true for **num**; at the end of **main()**, memory for **num** is automatically deallocated.

Automatic Memory Allocation

- While creating arrays through this style, length must be provided at **compile time**.
- Memory is also handled automatically (same as before).

```
1  #include<iostream>
2
3  int main()
4  {
5      int arr[10]{};
6
7      {
8          double constants[3]{ 3.14, 3.0e8, -273.15 };
9
10         } // "constants" cannot be accessed outside this block.
11
12         std::cout << constants << '\n';
13
14         return 0;
15     }
```

Problems with Automatic Memory Allocation

- Because the length has to be known at compile time...
 - You would either make very large arrays – ***wasted/unused memory***.
 - On the flip side, you may end up needing more memory – ***going out of bounds***.
- ***Stack memory is limited***, so very large arrays cannot be created.
 - Stack is the memory segment where automatic memory allocation happens.
 - More on this later.

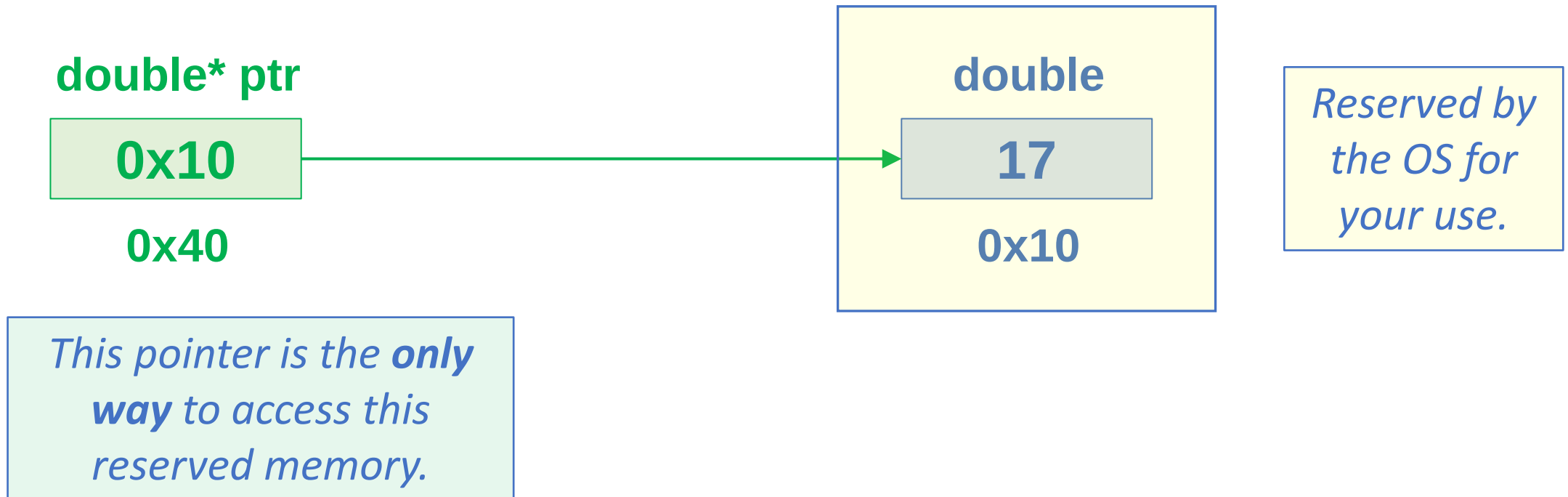
Dynamic Memory Allocation

Dynamic Memory Allocation

- *Dynamic* implies something that happens at **runtime**.
 - **Dynamic memory allocation** is a way for **running programs** to request memory from the OS when needed.
- If we don't know the length of the array at compile time, use **dynamic memory allocation**.
- *Let's start with dynamically allocating variables.*

Dynamic Memory Allocation

```
double* ptr = new double{ 17 } ;
```



Dynamic Memory Allocation

```
double* ptr{ new double{ 17 } };
```

- Note the **new** keyword.
- Note that a pointer is returned.
 - Its value is the address of the memory that the OS has reserved for you.
 - Dynamically allocated memory can **only** be accessed through these pointers.

Dynamic Memory Allocation

```
double* ptr{ new double };
```

- Note that you can allocate memory, **without initialization** as well.

Dynamic Memory Allocation

- There is a catch though...

You're responsible for deallocating/freeing this memory!

```
double* ptr{ new double{ 17 } }; // allocation  
delete ptr; // deallocation
```

Dynamic Memory Allocation

*We already
saw this...*

```
double* ptr{ new double{ 17 } };
```

double* ptr

0x10

0x40

double

17

0x10

*Reserved by
the OS for
your use.*

*This pointer is the **only**
way to access this
reserved memory.*

Dynamic Memory Allocation

Note

You're using the same pointer for deallocation.

delete ptr; // deallocation

- **delete ptr** is essentially **delete 0x10**

double* ptr

0x10

0x40



???

???

0x10

OS is now free to use this memory for other applications (or maybe your own application later on).

Dynamic Memory Allocation

Note

You're using the same pointer for deallocation.

delete ptr; // deallocation

- **delete ptr** is essentially **delete 0x10**

double* ptr

0x10

0x40

ptr is left
dangling.

???

???

0x10

OS is now free to use this memory for other applications (or maybe your own application later on).

Dynamic Memory Allocation

```
delete ptr; // deallocation  
ptr = nullptr; // set to nullptr
```

- Set the pointer to **nullptr** once you've freed the memory that it was pointing at.

What does it mean to delete ptr; ?

Some common misunderstandings...

What does it mean to **delete ptr**; ?

- The memory that **ptr** is pointing to is returned back to the operating system.
- The OS is then free to reassign that memory to another application (or to this application again later).
- The delete operator does not actually **delete or remove** anything.
 - The number of available memory locations does not shrink.
 - In fact, the pool of **free** memory has increased because of deallocation.

What does it mean to `delete ptr` ?

- Also, we are not deleting the pointer variable, `ptr`.
 - You can still work with `ptr` as long as it's in scope.
- Once you have performed **`delete`**, you can have `ptr` point to some other entity.
 - `ptr = &another_entity;`
- You also saw that it could be set to `nullptr`.
 - `ptr = nullptr;`
- You can dynamically allocate memory again.
 - `ptr = new int{-13};`

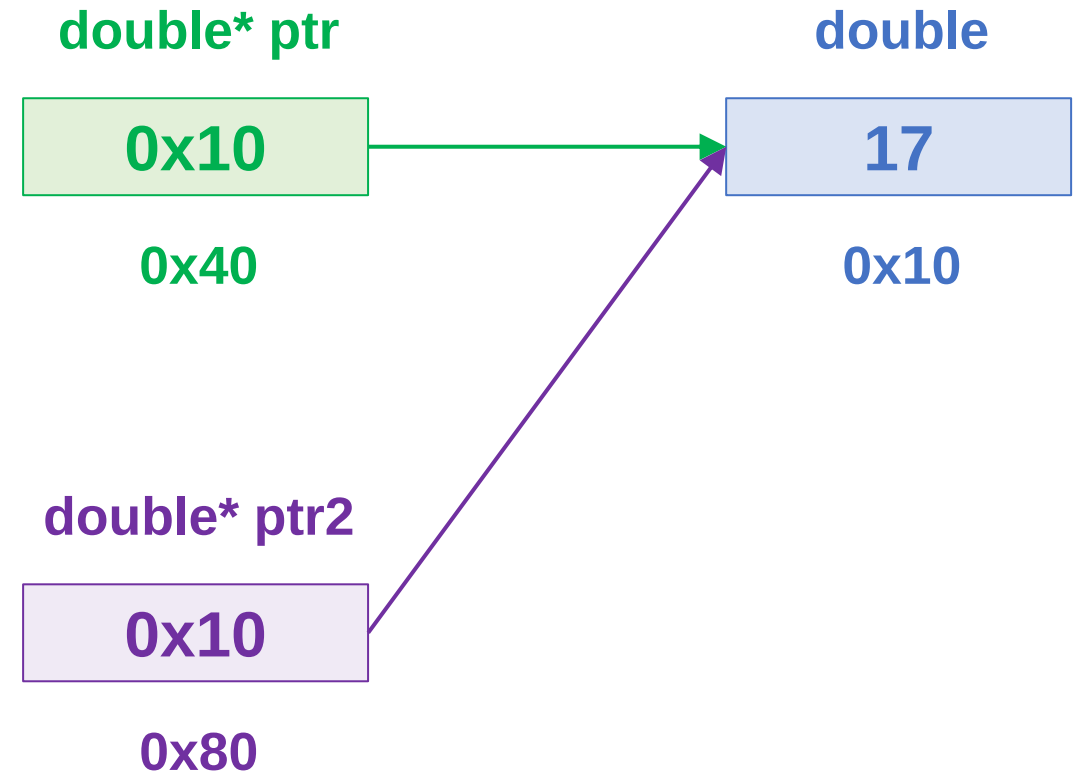
An Implication of delete

An Implication of delete

- Couple of questions...
- What if you had two (or more) pointers pointing at the same memory address, and that address gets deallocated?
- Similarly, what if you had two (or more) pointers pointing at the same variable, and that variable goes out of scope?

```
double* ptr{ new double{ 17 } };
```

```
double* ptr2{ ptr };
```



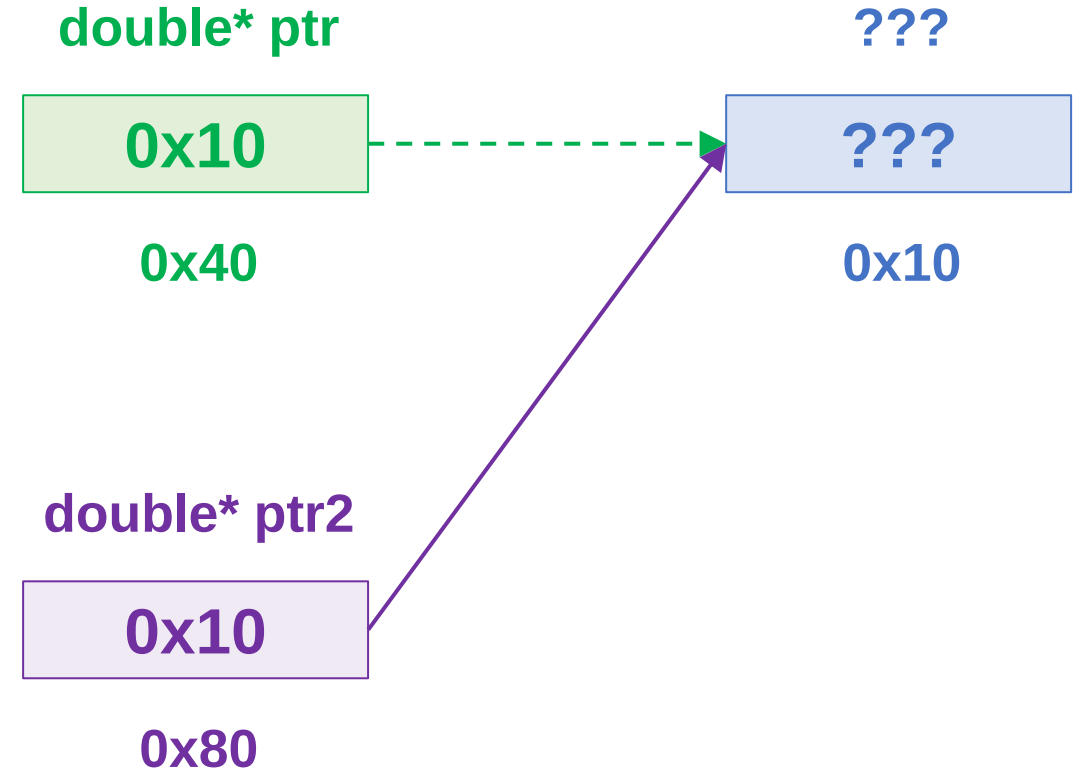
```
double* ptr{ new double{ 17 } };
```

```
double* ptr2{ ptr };
```

```
// ptr is left dangling
```

```
delete ptr;      // delete 0x10
```

What about **ptr2** though?



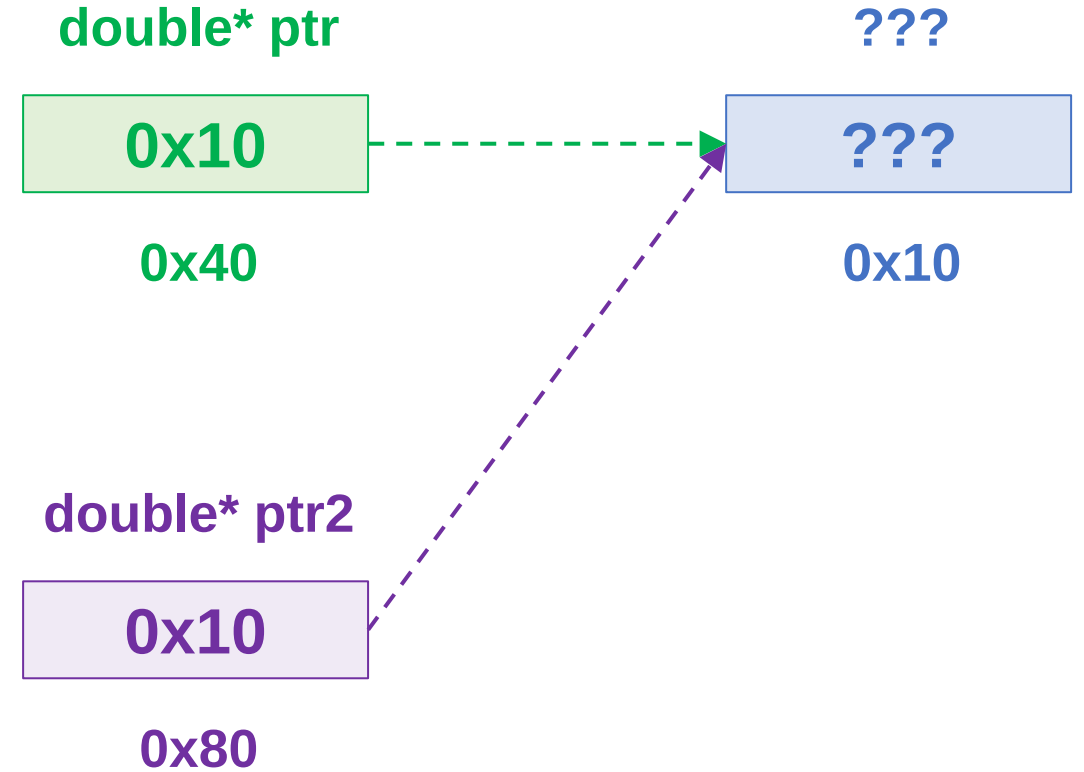
```
double* ptr{ new double{ 17 } };
```

```
double* ptr2{ ptr };
```

```
// ptr is left dangling
```

```
delete ptr;      // delete 0x10
```

What about **ptr2** though?
Also left in a dangling state.



```
double* ptr{ new double{ 17 } };
```

```
double* ptr2{ ptr };
```

```
// ptr is left dangling
```

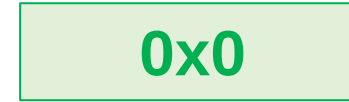
```
delete ptr;      // delete 0x10
```

```
ptr = nullptr;
```

```
ptr2 = nullptr;
```

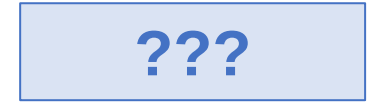
Set both to nullptr.

double* ptr



0x40

???



0x10

double* ptr2



0x80

Recommendations to Avoid this Issue

- Try to avoid having multiple pointers pointing at the same memory location.
- Set dangling pointers to **nullptr**.

Self-study

Go through the program.

An Implication of delete

```
1  #include <iostream>
2
3  int main()
4  {
5      int* ptr{ new int{} }; // dynamically allocate an integer
6      int* otherPtr{ ptr }; // otherPtr is now pointed at that same memory location
7
8      delete ptr; // return the memory to the operating system. ptr and otherPtr are now dangling pointers.
9      ptr = nullptr; // ptr is now a nullptr
10
11     // however, otherPtr is still a dangling pointer!
12
13     return 0;
14 }
```

Dynamically Allocating Arrays

Choosing lengths at runtime...

Dynamically Allocating Arrays

new[]
delete[]

- Lengths of dynamically allocated arrays **can be chosen at runtime**.
 - This is unlike fixed arrays.

```
int len{};
```

```
std::cin >> len;
```

```
int* arr{ new int[len]{} };
```

```
delete[] arr;
```

```
arr = nullptr;
```

Dynamically Allocating Arrays

new[]
delete[]

- The pointer is pointing at the first element of the dynamic array.
- With dynamically allocated arrays, you again need to **keep track of the length** using a separate variable.
 - **sizeof()** will just give you the size of the pointer.
- Dynamically allocated arrays can have **much larger lengths** than fixed arrays.
 - This is because heap memory is much larger than stack memory (*later in the lecture*).

Dynamically Allocating Arrays

new[]
delete[]

- A natural question comes up.
- How does **delete[]** know how much memory to deallocate?
 - We don't pass the size of the dynamic array to it.
- **new[]** keeps track of how much memory was reserved, so that **delete[]** can deallocate the proper amount.
 - This information, however, is hidden away from us.

Stack and Heap

Stack and Heap

- There are two types of memory that we should know about (there are others as well).
 - Stack
 - Heap
- **Stack** is used when you rely on **automatic memory allocation**.
- **Heap** is used when you rely on **dynamic memory allocation**.
- Each type of memory has its own pros and cons.

Stack

Automatic Memory Alloc.

- Accessing stack memory is fast and efficient.
- Stack is limited in memory.
 - Generally, it's **1 to 2 MB**.
 - If you try exceeding the limit → *program crash*.
- Two examples to accomplish this.
 - Mess around with the array lengths.
- Lengths have to be specified at compile time for arrays on stack.

```
13 int main()
14 {
15     // recursiveFunction(0);
16
17     char arr[1000 * 1000 * 3] {};
18     std::cout << static_cast<void*>(arr) << '\n';
19
20     return 0;
21 }
```

```
1 #include <iostream>
2
3 void recursiveFunction(int i)
4 {
5     char arr[1000 * 100] {};
6     int size { sizeof(arr) };
7
8     std::cout << "Recursive Call # " << i << std::endl;
9     std::cout << "Approx. Stack Memory used = "
10         << (size * (++i)) / 1.0E3
11         << " kB\n" << '\n';
12
13     recursiveFunction(i);
14 }
15
16 int main()
17 {
18     recursiveFunction(0);
19
20     return 0;
21 }
```

Heap

Dynamic Memory Alloc.

- Accessing heap memory is slower compared to stack.
 - Memory is accessed through a pointer.
 - Pointer takes you to the memory, and then you dereference it (**two steps**).
- Heaps, generally, have a lot of memory.
 - Can be gigabytes in size.

Heap

Dynamic Memory Alloc.

- May happen that you don't have enough memory in the heap.
 - If memory allocation fails, an exception is raised, and your program will crash.
- You can allocate memory with this modification.

```
int* arr{ new (std::nothrow) int[len]{} };
```

- Instead of throwing an exception, a null pointer is returned if memory allocation fails.
 - `arr ← nullptr`
- **while**(arr == `nullptr`), keep taking input from the user.

Memory Leaks

Memory Leaks

- Dynamically allocated memory **is not** automatically deallocated.
- Note that this memory will stay reserved for you until,
 - You explicitly deallocate it.
 - Your program or application ends (*your OS will reclaim that memory*).
- But what if you forget to deallocate the dynamic memory, and your program is still running?
 - *Basically, what if you forget to **delete ptr**.*

```
void createArray(int length)
{
    int* ptr{ new int[length]{} };
}
```

ptr goes out of scope at the end of this function call.

```
int main()
{
    int len{};
    std::cin >> len;

    createArray(len);

    // More code...

    return 0;
}
```

Side question: Is **ptr** a dangling pointer?

NO!

Recall

*The **dynamically allocated memory** will stay reserved for you until,*

- *You explicitly deallocate it.*
- *Your program ends.*

```
void createArray(int length)
{
    int* ptr{ new int[length]{} };
}
```

ptr goes out of scope at the end of this function call.

```
int main()
{
    int len{};
    std::cin >> len;

    createArray(len);

    // More code...

    return 0;
}
```

*When you return back to **main()**, you cannot access the memory that you just reserved.*

- You've **LOST** the address.*
- Also can't explicitly deallocate it now.*

The OS would also not use it for any other purpose.

*Available free memory has been reduced –
MEMORY HAS BEEN LEAKED.*

Memory Leaks

```
int* ptr{ new int[length]{} };
```

- Understand that **ptr** is in the stack memory.
- So before **ptr** goes out of scope, make sure that,
 - Explicitly deallocate the memory using **delete ptr** (or **delete[] ptr**) or,
 - Hold on to that address.
 - Copy/return **ptr**'s value to another pointer that would still be in scope.

*Why are these leaks?
How to resolve them?*

```
1 | int value = 5;  
2 | int* ptr{ new int{} }; // allocate memory  
3 | ptr = &value; // old address lost, memory leak results
```

More Memory Leaks

```
1 | int* ptr{ new int{} };  
2 | ptr = new int{}; // old address lost, memory leak results
```

This is what a memory leak could look like...

<https://www.youtube.com/watch?v=iEn0ozP-jxc>

- 7:08 minute mark.

Motivation for Learning Pointers

- Understanding how arrays work.
- Allowing us to use dynamically allocated memory from the heap.
- **<LOCKED>**
- etc...

Array of Pointers

Arrays of pointers

- Pointers to pointers have a few uses. The most common use is to dynamically allocate an array of pointers:
- `int** array { new int*[10] }; // allocate an array of 10 int pointers`

Two-dimensional dynamically allocated arrays

- First, we allocate an array of pointers (as per above).
- `int** p{ new int*[4] }; // allocate an array of 4 int pointers — these are our rows`
- Then we iterate through the array of pointers and allocate a dynamic array for each array element.
- `for (int count { 0 }; count < 4; ++count)`
- `p[count] = new int[3]; // these are our columns`
- Our dynamic two-dimensional array is a dynamic one-dimensional array of dynamic one-dimensional arrays!

